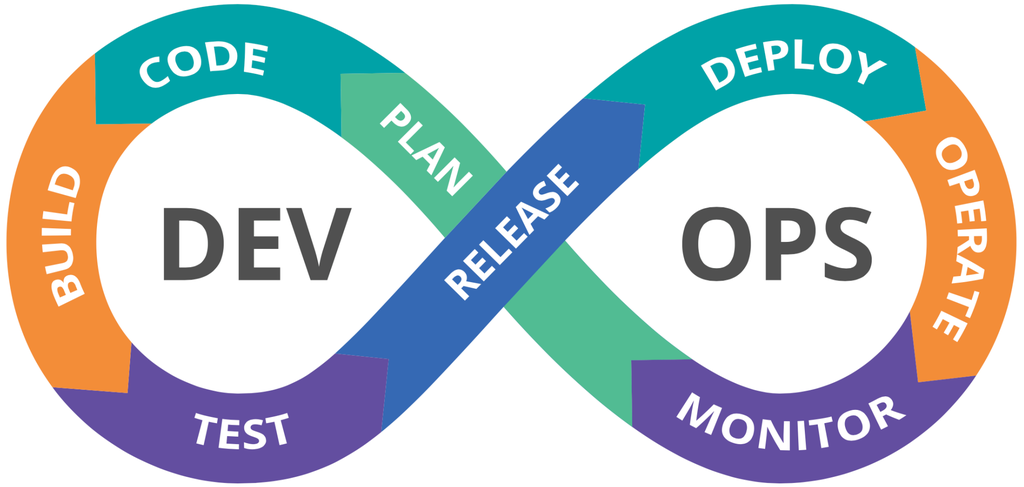
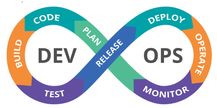


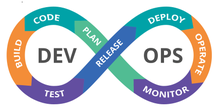
Tehničke prakse za uspostavljanje povratnih tokova
Povratne informacija kao podrška za bezbednu isporuku

predavanja su bazirana i slike preuzete iz knjige
*“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”*
Gene Kim, Jez Humble, Patrick Debois, John Willis



Tehničke prakse za uspostavljanje povratnih tokova

- Tehničke prakse koje omogućavaju uspostavljanje kvalitetnih povratnih informacija podrazumevaju:
  - kreiranje odgovarajuće telemetrije koja omogućava da se problemi uoče i reše
  - korišćenje telemetrijskih podataka za bolje predviđanje ponašanja sistema i dostizanja ciljeva
  - **obezbeđivanje povratnih informacija koje omogućavaju razvojnim i operativnim timovima da bezbedno isporučuju i instaliraju softver**
  - integrisanje korisničkih iskustava i povratnih informacija u rad razvojnih timova
  - obezbeđivanje povratnih informacija za povećanje kvaliteta putem revizije koda od strane kolega i programiranja u parovima



“Strah” od deploymenta

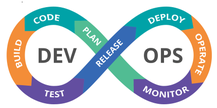
- U vrlo dinamičkim okruženjima pokazalo se da odvojeni timovi za razvoj, za testiranje i deployment (Ops) nisu dobro rešenje jer je ceo proces prespor

- U integrisanim timovima svi članovi, pa tako i programeri, članovi razvojnog tima dobijaju ulogu da mogu da isporuče kod (izvrše deplyment na produkciju).

  - desio se paradoks - iako su se često žalili da se Ops inženjeri “boje” da brzo instaliraju novoisporučeni kod, kada su dobili istu priliku, i sami programeri su zazirali od brzih isporuka koda

- Postojanje dobrih i pouzdanih povratnih informacija iz produkcionih okruženja utiče na to da se ovaj problem otkloni i ceo proces isporuke novih *feature-a* učini sigurnijim i bržim.

- Ovde opet ključnu ulogu mogu imati telemetrijski podaci iz same aplikacije na produkcionim okruženjima



Korišćenje telemetrijskih podataka kako bi se isporuka učinila sigurnijom

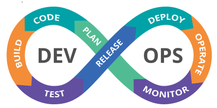
- Produkciona telemetrija mora aktivno da se nadzire kada god se radi isporuka nove verzije na produkciju. Ovo omogućava da bilo ko (Dev ili Ops) brzo utvrdi da li aplikacije na produkciji rade na očekivani način i nakon instaliranja nove verzije.

- Pri tome se najviše treba fokusirati na telemetrijske podatke koji su povezani upravo sa onim *feature*-ima koji su novi ili koji su menjani. Na taj način možemo se uveriti da servis koji smo instalirali nije slučajno “pokvaren” ili još bitnije da nova verzija našeg servisa ne ruši neke druge.

- Ako nove izmene ne rade kako treba brzo se vraćamo na prethodnu, funkcionalnu verziju (roll back), a radi se analiza problema koji je nastao sa novom verzijom.

  - može se uraditi i *fix-forward (*brza ispravka greške i puštanje na produkciju, ali poštujući kompletan proces pipeline-a)

  - zgodna opcija je ako postoje ranije spominjani “prekidači” za uključivanje i isključivanje određenih funkcionalnosti



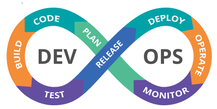
Raspodela dužnosti “dežurnih” između članova operativnog i razvojnog tima

- Čak i kada instaliranje prođe bez problema, u kompleksnim sistemima se uvek mogu očekivati iznenadni incidenti, a problem je kada se oni dese vikendima ili u noćnim satima.

- Ukoliko se ovi problemi ne reše oni postaju trajni problem operativnih timova, pogotovo ako se ta informacija ne propagira iz operativnih timova u razvojne

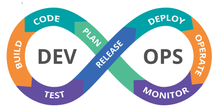
- Kako bi se ovo izbeglo jedna od tehnika (organizaciona) je da se svi uključe u dužnosti “dežurnih” koji nadziru rad aplikacije i pružaju podršku klijentima.

- Na ovaj način svi članovi tima “dožive” i postaju svesni problema sa kojima se mogu suočavati članovi operativnih timova.



Članovi razvojnog tima “prate” svoj rad duž lanca produkcije

- Contextual inquiry - jedno od ključnih koncepata UX, kada timovi koji su radili razvoj posmatraju kako stvarni klijenti koriste aplikaciju u svom redovnom okruženju.
  - Ovo često dovodi do toga da programeri spoznaju neverovatne načine kako neko koristi aplikaciju, ali isto tako da vide da ono što su napravili zahteva od korisnika da za osnovni zadatak svog posla obavi nekoliko desetina klikova
  - Ovo može dovesti do temeljne promene kako programeri pristupaju razvoju programa
- Isti princip treba primenjivati i na “interne korisnike” dakle druge članove razvojnog lanca
  - Na taj način programeri mogu da shvate kako način implementacije koji su izabrali može da proizvede probleme operativnim timovima



Članovi razvojnog tima inicijalno sami upravljaju svojim servisom u produkciji

- Čak i kada članovi razvojnih timova rade na sistemima sličnim produkcionim, operativni timovi često mogu da dožive vrlo problematične isporuke na stvarni produkcioni sistem.
- Google-ov pristup je da razvojni timovi inicijalno sami upravljaju servisima koje su razvili i instalirali u produkciona okruženja. Tek naknadno taj servis prelazi u nadležnost operativnih timova.
  - moraju se postaviti minimalni zahtevi za lansiranje proizvoda kako bi se izbeglo da timovi u produkciju puštaju neproveren kod
    - Broj defekata i nivo važnosti
    - Tip i frekvencija alarma - da li aplikacija generiše toliku količinu alarma da ih je nemoguće obraditi?
    - Pokrivenost monitoringom - da li imamo dovoljno dobar nadzor da možemo brzo detektovati problem i oporaviti sistem
    - Arhitektura sistema - da li je takva da podržava česte isporuke novih modula/verzija
    - Proces isporuke - da li je uspostavljen predvidiv, pouzdan i automatizovan sistem isporuke
    - “Higijena” proizvoda - da li su uspostavljene dobre proizvodne navike (provera koda, testovi i sl).
  - Inicijalno sami razvojni timovi će morati odgovoriti na sve alarme i zahteve i žalbe korisnika.